

```

    <button onClick={() => setCount(count + 1)}>
      Click me
    </button>
  </div>
);
}

```

useState is a hook that lets you add React state to function components. If you write a function component and realise you need to add some state to it, until now you had to convert it into a class. So if we wanted to define a state without using hooks, we would likely have a state variable, initialise it in the constructor and update the state on a button click by calling the *setState* function. With hooks, you can avoid this process altogether.

Let's say we have a functional component. We can now add state to it using hooks. For hooks, we don't need to change components into a class. We import the *useState* hook and use it inside the function component. We also add some local state to it. React will preserve the state between renders and return a pair consisting of the current state value, in this case *count*, and a function that updates the value, in this case, *setCount*; you can use the function instead of *this.setState*, as shown in the code. We can just call *setCount* and pass the count value that we want. We can use this state hook multiple times in a component.

useEffect

We move on to the second basic hook, *useEffect*. React does not allow side effects such as mutations, subscription timers, etc, to be implemented in the render phase. If we do so, we can get confusing bugs and inconsistencies in the UI. To avoid that, we use *useEffect*. *useEffect* will run after a render, but React provides a way of conditional rendering too, which we will see later. Let's say you want to edit the HTML document title every time the *count* value changes. Without hooks, you will need to specify the following two functions — *componentDidMount* and *componentDidUpdate* -- and add the same piece of code in both the functions, as shown below.

```

class App extends React.Component {

  constructor() {
    super();
    this.state = {
      count: 0
    };
  }

  componentDidMount() {
    document.title = `You clicked ${this.state.count}
times`;
  }
}

```

```

  componentDidUpdate() {
    document.title = `You clicked ${this.state.count}
times`;
  }

  render() {
    return (
      ...
    );
  }
}

```

Using hooks you can simplify this with only one call to *useEffect*, as shown below; its behaviour is exactly the same as that of the two methods together.

```

function MyCounter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  });

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}

```

Now let's say we want to update the document only when the *count* is different. Without hooks, we would use *componentDidUpdate* that has *prevProps* and *prevState* parameters. We'll add the condition for checking the count, and only when it is different will we update the document title.

```

class App extends React.Component {

  constructor() {
    super();
    this.state = {
      count: 0
    };
  }

  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
      document.title = `You clicked ${this.state.count}
times`;
    }
  }
}

```